

Inventory & Stock Management System

A Console-Based Python Application for Product, Stock and Sales Management

Course: CSE1021 - Introduction to Problem Solving

Submission: VITyarthi - Build Your Own Project

GitHub Repository: github.com/shantanu5bhadra-bit/inventory-management-system

Project Report

2. Introduction

This report documents the design, implementation, and evaluation of an Inventory & Stock Management System developed in Python as the "Build Your Own Project" submission for the course CSE1021 - Introduction to Problem Solving. The project applies core problem-solving and programming concepts covered in the course - control structures, functions, data structures, file handling, validation, and modular program design - to a realistic, practical scenario.

The application is a menu-driven command-line program that allows a shop owner to manage a catalog of products, track stock movements, record sales, and generate operational reports, all backed by reliable, persistent JSON-based storage.

3. Problem Statement

Small shop owners and small-scale retailers frequently manage stock, sales, and reordering manually using notebooks or basic spreadsheets. This leads to common problems: stockouts of fast-moving items, overstocking of slow-moving items, no easy way to track revenue, and no systematic way to know when a product needs to be reordered. There is a need for a simple, reliable, offline tool that lets a shop owner track products, stock levels, and sales without requiring internet access, a database server, or specialised hardware.

4. Functional Requirements

The system implements three major functional modules:

Module 1: Product Management

- Add a new product with name, category, price, quantity and reorder level
- View all products in a formatted table
- Search products by name (partial, case-insensitive match)
- Update an existing product's details
- Delete a product from the catalog

Module 2: Inventory Operations

- Stock In - increase a product's quantity (e.g. on restock)
- Stock Out - decrease a product's quantity, with validation against available stock
- Automatic low-stock detection against each product's configurable reorder level

Module 3: Sales & Reporting

- Record a sale (validates stock availability, deducts stock, computes total)
- Generate a formatted receipt for each sale
- Inventory valuation report (current stock value)
- Sales summary report (quantity sold and revenue per product)

- Top-selling products ranking

Input/output structure: every operation follows a consistent pattern - the CLI prompts for input, the relevant manager module validates and processes it, the result (or a clear error message) is displayed, and the action is logged and persisted to disk.

5. Non-Functional Requirements

Requirement	How it is addressed
Reliability	Atomic JSON writes (write to a temp file, then <code>os.replace</code>) so a crash mid-save can never corrupt
Usability	Clear numbered menu, consistent prompts, formatted tables, and human-readable error messages
Error Handling	A custom <code>ValidationError</code> type plus <code>try/except</code> around every menu action in <code>main.py</code> ; invalid input
Logging / Monitoring	Every significant action (add, update, delete, stock change, sale) is timestamped and written to l
Maintainability	Code is split into 8 single-responsibility modules with docstrings, so any one feature can be mod
Performance	Product lookups and in-memory list operations are efficient for the expected catalog sizes of a s

6. System Architecture

The system follows a layered architecture. The CLI layer (`main.py`) handles all user interaction and delegates every operation to the business logic layer. The business logic layer (four manager modules) implements the actual problem-solving logic and depends on a shared support layer (data models, validators, logging) and a persistence layer that performs atomic reads and writes to JSON files. This separation means the storage mechanism, validation rules, or CLI presentation can each be changed independently without breaking the others.

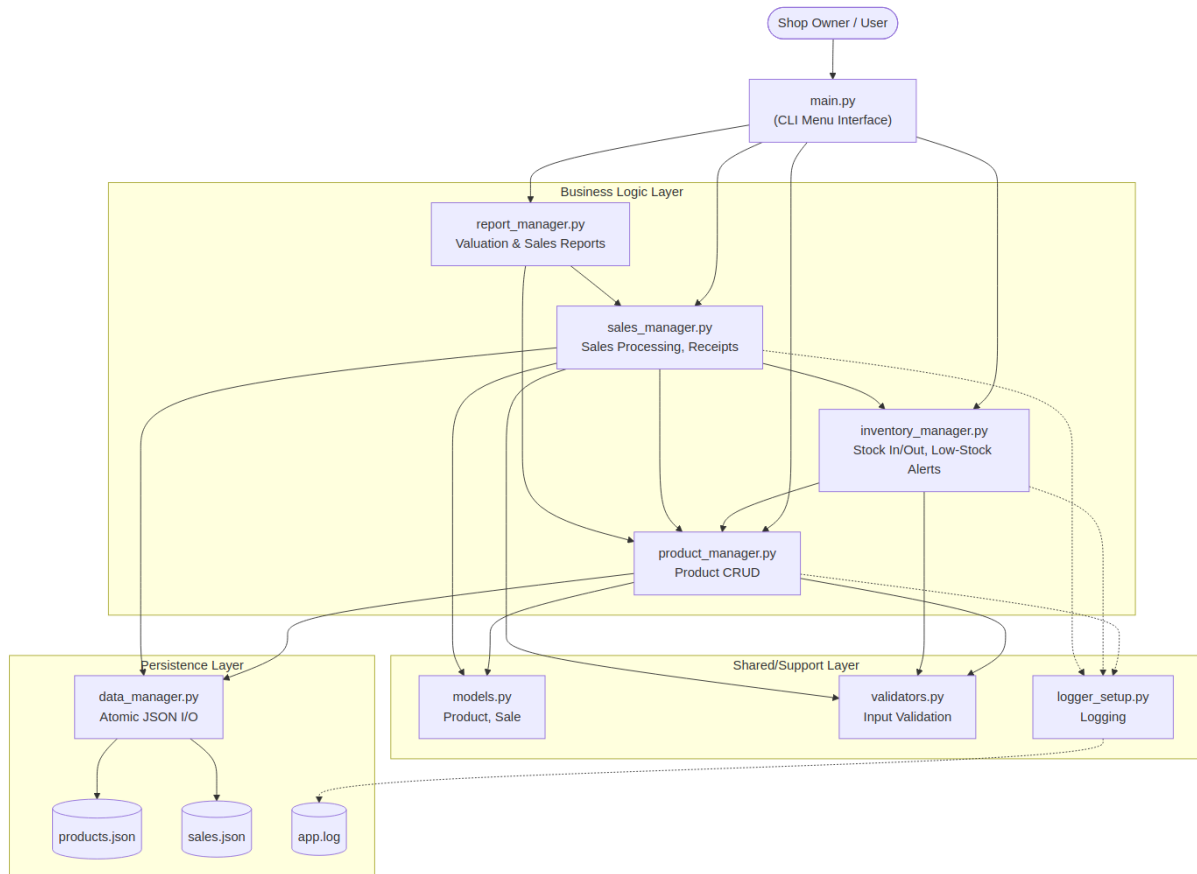


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Process Flow / Workflow Diagram

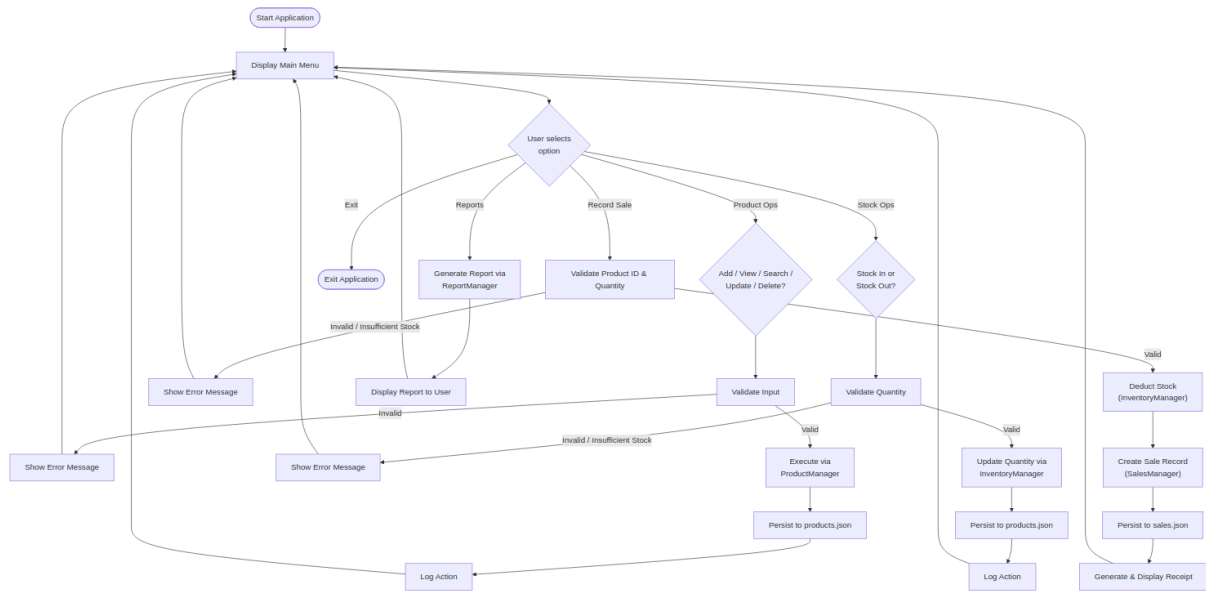


Figure 2: Process Flow / Workflow Diagram

7.2 Use Case Diagram

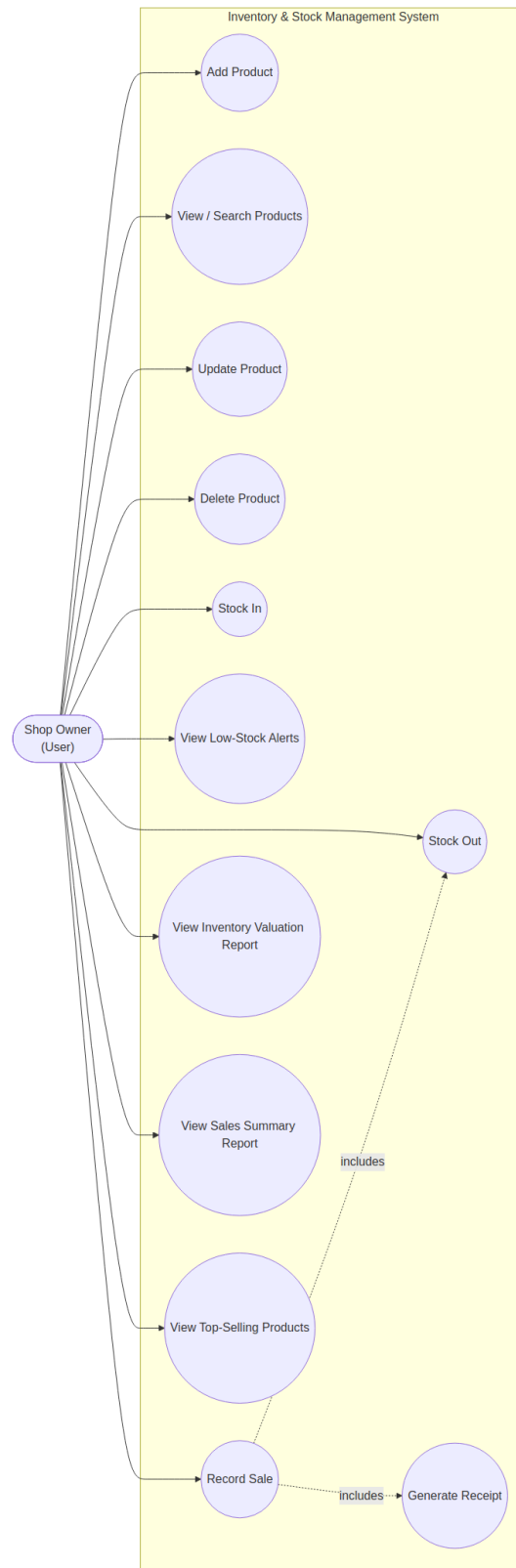


Figure 3: Use Case Diagram

7.3 Class / Component Diagram

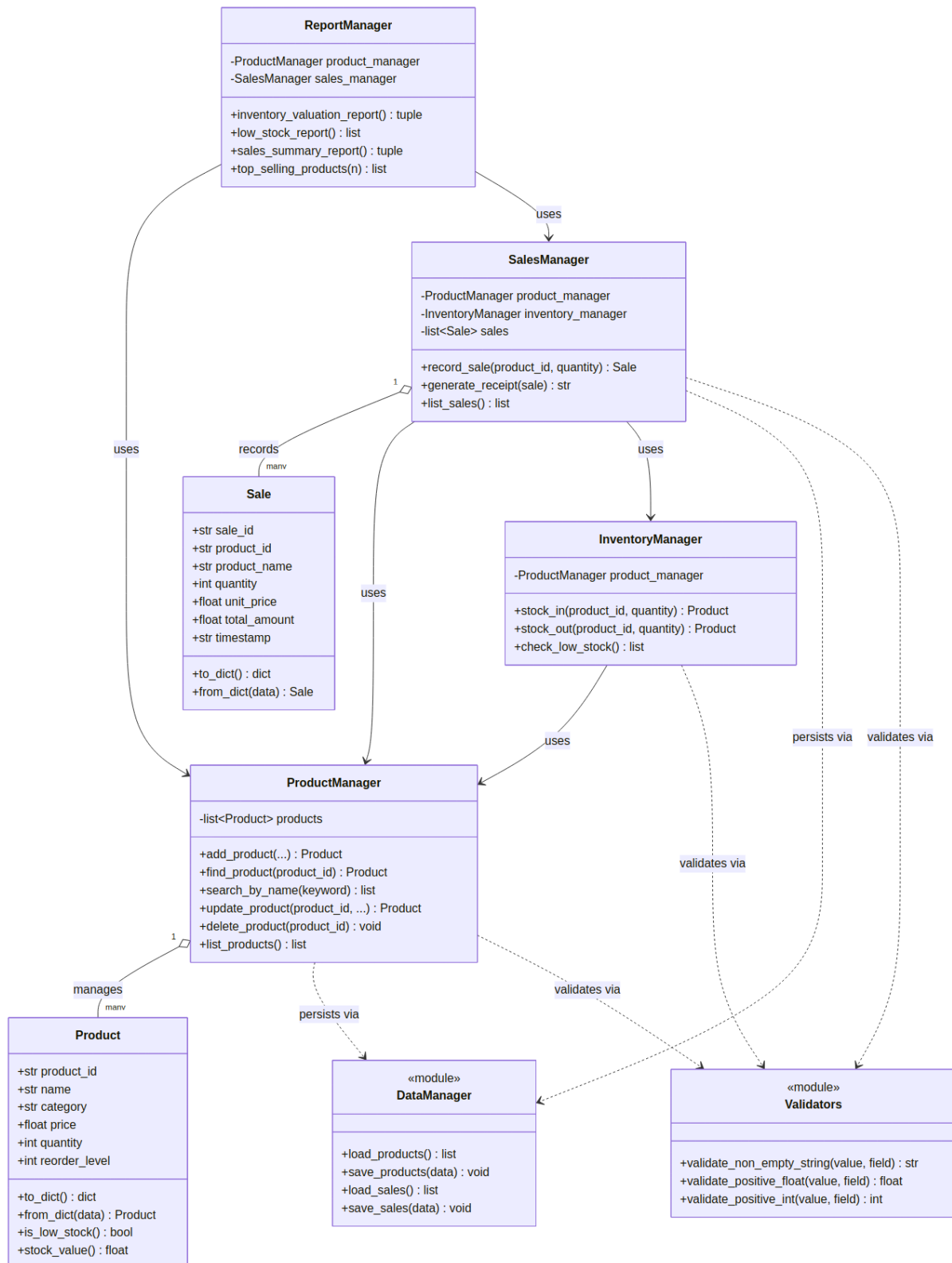


Figure 4: Class Diagram

7.4 Sequence Diagram - Record a Sale

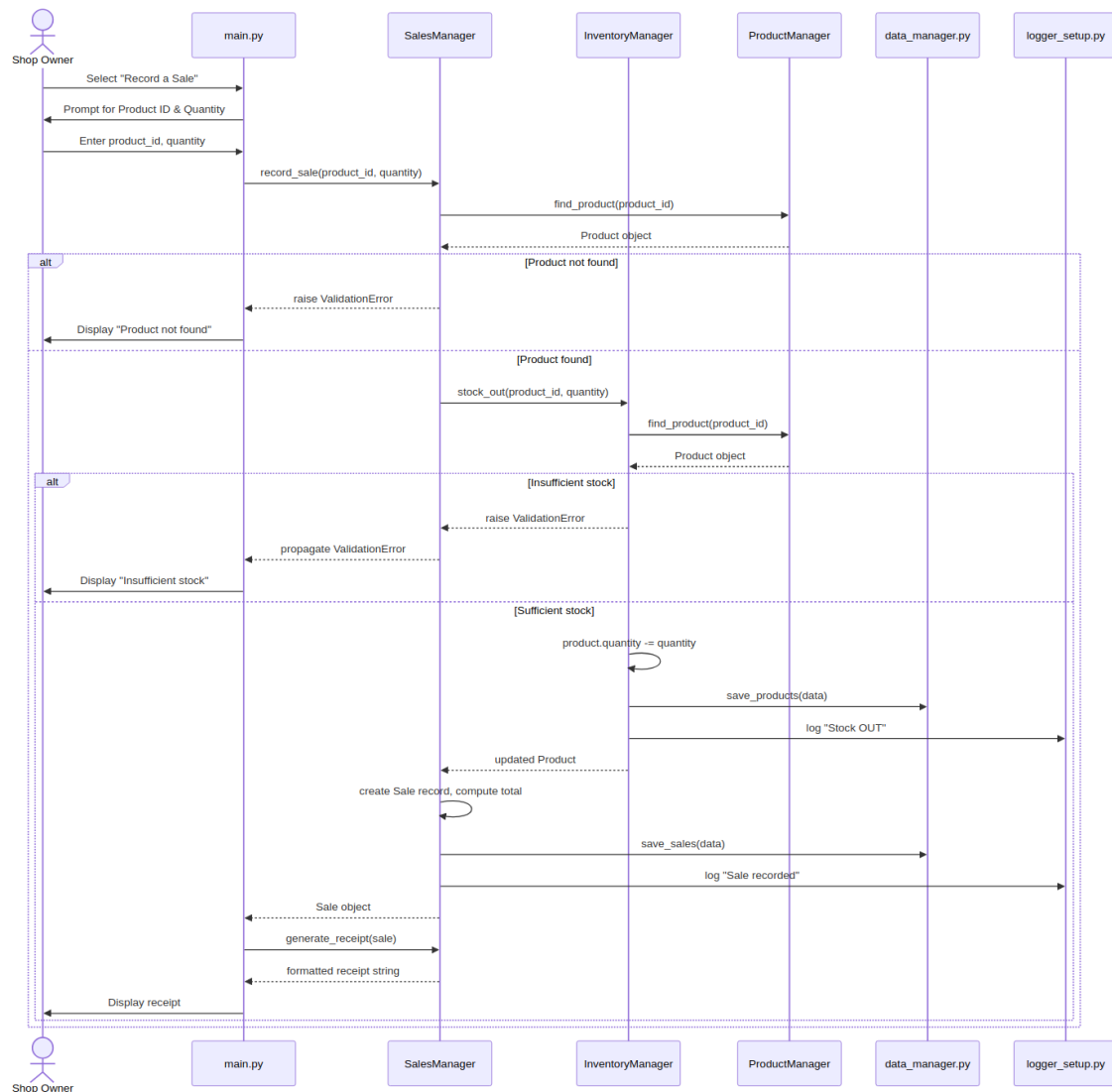


Figure 5: Sequence Diagram for the 'Record Sale' operation

7.5 Database / Storage Design

This project uses flat-file JSON storage rather than a relational database, so a traditional ER diagram does not apply. The two data files and their schemas are:

File	Fields
data/products.json	product_id, name, category, price, quantity, reorder_level
data/sales.json	sale_id, product_id, product_name, quantity, unit_price, total_amount, timestamp

8. Design Decisions & Rationale

- **JSON over a relational database:** the project targets a single small shop with modest data volumes; JSON gives simple, human-readable, dependency-free persistence appropriate for an introductory course, while the `data_manager` module isolates this choice so it could be swapped for a real database later without changing any business logic.
- **Dataclasses for models:** `Product` and `Sale` are implemented as Python dataclasses, giving clear, typed structures with automatic `to_dict()/from_dict()` conversion for JSON serialization.
- **Atomic writes:** every save writes to a temporary file and then uses `os.replace()`, which is atomic on both POSIX and Windows, so an interrupted write can never leave a half-written, corrupted data file.
- **Centralized validation:** all input validation lives in `validators.py` and raises a single `ValidationError` type, so every module and the CLI can handle bad input identically.
- **Layering managers by responsibility:** `InventoryManager` depends on `ProductManager`, and `SalesManager` depends on both, mirroring the real-world dependency between stock levels and sales - a sale cannot succeed unless inventory allows the stock deduction.

9. Implementation Details

The project is implemented in pure Python 3 (standard library only - `json`, `logging`, `dataclasses`, `uuid`, `datetime`, `os`, `unittest`). It consists of 8 modules plus the CLI entry point:

File	Responsibility
<code>main.py</code>	CLI menu loop; wires all managers together; catches and displays all errors
<code>modules/models.py</code>	<code>Product</code> and <code>Sale</code> dataclasses with JSON (de)serialization
<code>modules/data_manager.py</code>	Atomic JSON load/save for both data files
<code>modules/validators.py</code>	Reusable input validation functions and <code>ValidationError</code>
<code>modules/logger_setup.py</code>	Centralized logging configuration (<code>logs/app.log</code>)
<code>modules/product_manager.py</code>	Product CRUD operations
<code>modules/inventory_manager.py</code>	Stock in/out and low-stock detection
<code>modules/sales_manager.py</code>	Sale recording and receipt generation
<code>modules/report_manager.py</code>	Valuation, sales summary and top-seller reports

Product IDs and Sale IDs are generated using truncated UUIDs (`uuid.uuid4()[:8]`) to guarantee uniqueness without requiring a central counter. Every write-path operation (add/update/delete product, stock in/out, record sale) immediately persists to disk, so no data is lost if the program is closed unexpectedly.

10. Screenshots / Results

```
=====
                          INVENTORY & STOCK MANAGEMENT SYSTEM
=====
1. Add Product
2. View All Products
3. Search Product by Name
4. Update Product
5. Delete Product
6. Stock In (add stock)
7. Stock Out (remove stock)
8. Record a Sale
9. View Low Stock Alerts
10. Inventory Valuation Report
11. Sales Summary Report
12. Top Selling Products
0. Exit
=====
```

Figure 6: Main menu on application start

```
=====
Enter your choice: 2
ID          Name          Category    Price    Qty    Reorder
-----
d0a355f4    Notebook    Stationary  80.00    50     10
f37c48ac    Pen         Stationary  10.00     8     10
=====
                          INVENTORY & STOCK MANAGEMENT SYSTEM
=====
```

Figure 7: Viewing all products after adding two items

```
=====
Enter your choice: 8
Enter product ID: d0a355f4
Quantity sold: 3
----- RECEIPT -----
Sale ID    : fd65e9fa
Date       : 2026-09-25 09:50:57
Product    : Notebook
Quantity   : 3
Unit Price : 80.00
Total      : 240.00
-----
=====
```

Figure 8: Receipt generated after recording a sale

```

Enter your choice: 9
LOW STOCK ALERT for the following products:
ID          Name          Category      Price      Qty      Reorder
-----
f37c48ac    Pen          Stationary    10.00      8        10

```

Figure 9: Low-stock alert for a product at/below its reorder level

```

Enter your choice: 10

ID          Name          Qty      Price      Value
-----
d0a355f4    Notebook      47      80.00     3760.00
f37c48ac    Pen           8       10.00      80.00

Total Inventory Value: 3840.00

```

Figure 10: Inventory valuation report

```

=====

Enter your choice: 11

ID          Name          Qty Sold  Revenue
-----
d0a355f4    Notebook      3        240.00

Total Revenue: 240.00

```

Figure 11: Sales summary report

11. Testing Approach

Automated unit tests are written with Python's built-in unittest framework, covering both modules that carry the system's core logic. Tests use isolated temporary JSON files (via monkey-patched file paths) so they never touch real application data.

- **tests/test_product_manager.py** (5 tests): adding a valid product, rejecting an empty name, rejecting a negative price, updating a product, deleting a product, and searching by name.
- **tests/test_inventory_and_sales.py** (6 tests): stock in increases quantity, stock out decreases quantity, stock out fails on insufficient stock, low-stock detection triggers correctly, recording a sale deducts stock and produces a correct receipt, and recording a sale with insufficient stock is rejected.

All 12 tests pass (verified with `python -m unittest discover tests -v`). In addition, the full CLI was manually exercised end-to-end (see Section 10), covering every menu option including deliberate invalid-input and insufficient-stock scenarios to confirm graceful error handling.

12. Challenges Faced

- Ensuring stock deduction and sale recording stay consistent even if an error occurs partway through - resolved by having SalesManager call InventoryManager.stock_out() first, so a failed stock check aborts the sale before any sale record is created.
- Preventing data corruption if the program is interrupted while saving - resolved with atomic temp-file-then-replace writes in data_manager.py.
- Keeping the CLI usable for non-technical users - resolved by centralizing all validation messages and wrapping every menu branch in a try/except so no stack trace is ever shown to the user.
- Environment setup on Windows (Python not initially installed/on PATH) - resolved by installing Python from python.org with the 'Add python.exe to PATH' option enabled.

13. Learnings & Key Takeaways

- Breaking a real-world problem into independent modules (product, inventory, sales, reporting) makes the logic far easier to reason about, test, and extend than one large script.
- Centralizing validation and error handling early prevents scattered, inconsistent error checks throughout the codebase.
- Simple techniques like atomic file writes meaningfully improve reliability without adding real complexity.
- Writing unit tests alongside the manager modules made it much faster to catch logic errors (e.g. in low-stock threshold comparisons) than manual testing alone.

14. Future Enhancements

- Migrate persistence from JSON files to a proper relational database (e.g. SQLite) for larger catalogs and concurrent access.

- Add a simple graphical interface (Tkinter) as an alternative to the CLI.
- Support multiple users/roles (e.g. cashier vs. manager) with different permissions.
- Add date-range filtering to the sales summary report and export reports to CSV/PDF.
- Add barcode scanning support for faster product lookup at checkout.

15. References

- Python 3 Standard Library Documentation - docs.python.org/3/
- Python 'json' module documentation - docs.python.org/3/library/json.html
- Python 'logging' module documentation - docs.python.org/3/library/logging.html
- Python 'dataclasses' module documentation - docs.python.org/3/library/dataclasses.html
- Python 'unittest' framework documentation - docs.python.org/3/library/unittest.html
- VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines